



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

VIT CHATBOT

Team Members:

20BDS0191 (MRIDUL MADNANI)
20BDS0266 (SUBHRONEEL BISWAS)

Report submitted for the
Final Project Review of

Course Code: CSE4022 – Natural Language Processing

Slot: E2+TE2

Professor: Dr. Priya G
SCOPE

Introduction

In recent years, we've become familiar with chatbots and how beneficial they can be for business owners, employees, and customers alike. Despite what we're used to and how their actions are fairly limited to scripted conversations and responses, the future of chatbots is life-changing, to say the least. The standard usage might not require more than quick answers and simple replies, but it's important to know just how much chatbots are evolving and how Natural Language Processing (NLP) can improve their abilities. This function holds plenty of rewards, really putting the 'chat' in the chatbot. This seemingly complex process can be identified as one which allows computers to derive meaning from text inputs. Put simply, NLP is an applied artificial intelligence (AI) program that helps the chatbot analyse and understand the natural human language communicated with the customers. Chatbots can understand the intent of the conversation rather than just use the information to communicate and respond to queries. Business owners are starting to feed their chatbots with actions to "help" them become more humanized and personal in their chats. It's incredible just how intelligent chatbots can be if you take the time to feed them the information they need to evolve and make a difference in the business. This intent-driven function will be able to bridge the gap between customers and businesses, making sure that the chatbot is something customers want to speak to when communicating with the business. We have built a chatbot that is able to answer any queries regarding VIT university. All the information on faculty, courses, transportation, events, etc. can be easily accessed through our chatbot. The final scope of this work is to design a chatbot that could help students in their campus life.

Literature Review Summary Table

<i>Authors and Year (Reference)</i>	<i>Title (Study)</i>	<i>Concept /Theoretical Model/ Framework</i>	<i>Methodology Used /Implementation</i>	<i>Dataset Details/ Analysis</i>	<i>Relevant Findings</i>	<i>Limitations/ Future Research/Gap Identified</i>
Sameera and John (2015)	Survey on Chatbot Design Techniques in Speech Conversation Systems	Design Techniques for Chatbots	Survey and Literature Review	Theoretical aspect for VOI chatbots	Various chatbot design techniques, strengths and weaknesses of each technique, and areas for improvement	Lack of empirical studies, limited focus on speech-based chatbots, need for user evaluation and testing
Dibitonto et al. (2018)	Chatbot in a Campus Environment: Design of LiSA, a Virtual Assistant to Help Students in Their University Life	LiSA chatbot framework for campus environments	Design and implementation of LiSA chatbot framework	Analysis of theoretical aspect for LiSA	LiSA chatbot framework can provide personalized assistance to students, improve student engagement and satisfaction, and reduce administrative workload for staff	Limited evaluation of LiSA framework, need for further testing and improvement

Reynald (2017)	Natural Language Processing using Python	Natural Language Processing concepts and techniques	Python programming and implementation of NLP algorithms	Theoretical aspects of Python with NLP	Overview of NLP concepts and techniques, and practical implementation on examples using Python	Limited focus on chatbot implementation, need for more in-depth discussion and analysis
Shakhovska et al. (2019)	Development of the Speech-to-Text Chatbot Interface Based on Google API	Speech-to-Text Chatbot Interface using Google API	Design and implementation of chatbot interface using Google API	Analysis of theoretical aspect for Assistants	Speech-to-Text chatbot interface can improve accessibility and ease of use for users, and reduce costs for developers	Limited evaluation of interface, need for more user testing and feedback
Ayanouz et al. (2020)	A Smart Chatbot Architecture based NLP and Machine Learning for Health Care Assistance	Chatbot architecture for healthcare assistance	Design and implementation of chatbot architecture using NLP and ML	Medical dataset with patient symptoms and diseases	Chatbot can provide accurate diagnosis and treatment recommendations, and reduce healthcare costs and workload	Limited evaluation on real-world patient data, need for further testing and refinement

Alemi and Ginsparg (2015)	Text Segmentation based on Semantic Word Embeddings	Text segmentation based on semantic word embeddings	Design and implementation of text segmentation algorithm	Theoretical aspects of NLP Segmentation concept	Semantic word embeddings can improve text segmentation accuracy, and enable more effective NLP tasks such as text classification and clustering	Limited evaluation on specific NLP tasks, need for more generalization and testing
Kulkarni et al. (2017)	Bank Chat Bot - An Intelligent Assistant System using NLP and Machine Learning	Chatbot architecture for banking assistance	Design and implementation of chatbot architecture using NLP and ML	Banking dataset with customer queries and responses	Chatbot can improve customer satisfaction, reduce workload for staff, and increase revenue for banks	Limited evaluation on real-world customer data, need for more user feedback and testing
Pradeep et al. (2022)	Voice Recognition Using Natural Language Processing	Voice recognition using NLP	Design and implementation of voice recognition system using NLP	Analysis of theoretical aspect for VOI Assistant bots.	Voice recognition using NLP can enable more intuitive and natural user interfaces for Applications, chatbots	Limited evaluation on specific applications, need for more testing and refinement

Li et al. (2006)	Sentence Similarity Based on Semantic Nets and Corpus Statistics	Sentence similarity based on semantic nets and corpus statistics	Design and implementation of sentence similarity algorithm	Theoretical aspects of NLP corpus concept	The proposed method outperformed existing methods in sentence similarity	The study did not evaluate the proposed method in a specific application domain
Haller & Rebedea, (2013)	Designing a Chatbot that Simulates an Historical Figure	Design of a chatbot that simulates an historical figure	The study presents the implementation of a chatbot that simulates the Romanian writer Mihail Sadoveanu, using a rule-based approach that takes into account the above-mentioned dimensions. The chatbot is designed to answer questions related to Sadoveanu's life, works, and opinions.	The study does not use any specific dataset, but rather relies on historical and literary sources related to Sadoveanu's life and work	The proposed chatbot effectively simulated the personality and characteristics of the historical figure	The study did not evaluate the chatbot's performance in a specific application domain

Objective of the project

The objective of creating a specific college chatbot using the above codes could be to provide a virtual assistant to students for answering their queries related to the college such as admission process, courses offered, fees structure, facilities available, important dates, etc. This chatbot can also provide personalized guidance and support to students by analyzing their academic performance, interests, and preferences, and recommending suitable courses, programs, or activities. The chatbot can also assist the college staff by answering frequently asked questions and providing timely updates to students and parents. Overall, the objective is to create a user-friendly and efficient communication channel between students and the college administration.

NLP bots generate responses based on user inputs. At times, constraining user input can be a great way to focus and speed up query resolution. So, when logical, falling back upon rich elements such as buttons, carousels or quick replies won't make the bot seem any less intelligent. To the contrary, besides the speed, rich controls also help to reduce users' cognitive load. Hence, they don't need to wonder about what is the right thing to say or ask. When in doubt, always opt for simplicity. The lack of a conversation ender can easily become an issue and you would be surprised how many NLP chatbots don't have one. If the user isn't sure whether or not the conversation has ended the bot might end up looking stupid or it will force you to work on further intents that would have otherwise been unnecessary. But the rule and choice-based bots don't have this issue. By considering the above points, we have created an NLP chatbot that is capable of answering most of the possible questions about VIT. We thought of all the possible scenarios in which our chatbot will be useful and designed it in an impeccable way.

Innovation component in the project

The project differs from others by incorporating specific college-related information and ensuring that the responses provided by the chatbot are accurate and up-to-date. It can also be designed to handle user inputs in natural language, making the conversation with the chatbot more intuitive and user-friendly. Additionally, the chatbot can be integrated with the college's website and social media channels to provide students and prospective students with easy access to information.

The routine experimental results of such a project can include metrics such as user satisfaction, response accuracy, and response time. The chatbot's performance can be evaluated by comparing its responses to the queries with the actual answers provided by the college. The project's success can be measured by the level of user engagement and feedback, as well as the reduction in the workload on college staff in handling routine queries.

The project of building a VIT chatbot using NLP and python code as a base, the innovation component is in the form of customizing the chatbot to cater to the specific needs and requirements of the college i.e. VIT. For example, the chatbot can be designed to answer queries related to admission procedures, course details, campus facilities, student life, and events, among other things.

Some possible innovations for this project include:

1. *Incorporated sentiment analysis*: The chatbot is trained to detect the sentiment of the user's input and respond accordingly. This can be useful in situations where the user is expressing frustration or dissatisfaction with the college.
2. *Implemented multi-lingual support*: The chatbot is trained to recognize and respond to user input in different languages. This can be useful in colleges with a diverse student population.
3. *Added personalized recommendations*: The chatbot is trained to provide personalized recommendations based on the user's interests and preferences, such as clubs or activities to join, classes to take, or resources to access.

Overall, these innovations can enhance the user experience and make the chatbot more useful and engaging for students.

Work done and implementation

Methodology

Chatbots work by using three classification methods:

1. Pattern matching

Pattern matching to analyse the text and produce a suitable response. The standard structure of these patterns is Artificial Intelligence Markup Language.

For example:

<pattern> Who is the developer? <pattern>

<template> Mridul Madnani and Subhroneel Biswas developed me as part of NLP project. <template>

Chatbot knows the answer because his or her name is part of an associated pattern. But for more advanced information, which is beyond the related pattern, the chatbot will need to use algorithms.

2. Algorithms

Algorithms reduce the number of classifiers and create a more manageable structure. In the following example, each class is assigned a score.

Term: “Good” (class: Greetings)

Term: “morning” (class: Greetings)

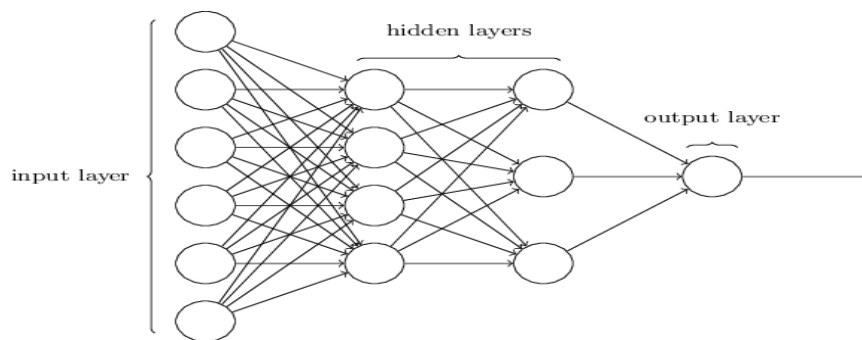
Classification: Greetings (score=2)

With the help of an equation, word matches are found for the given sentence, and this identifies the class with the highest match.

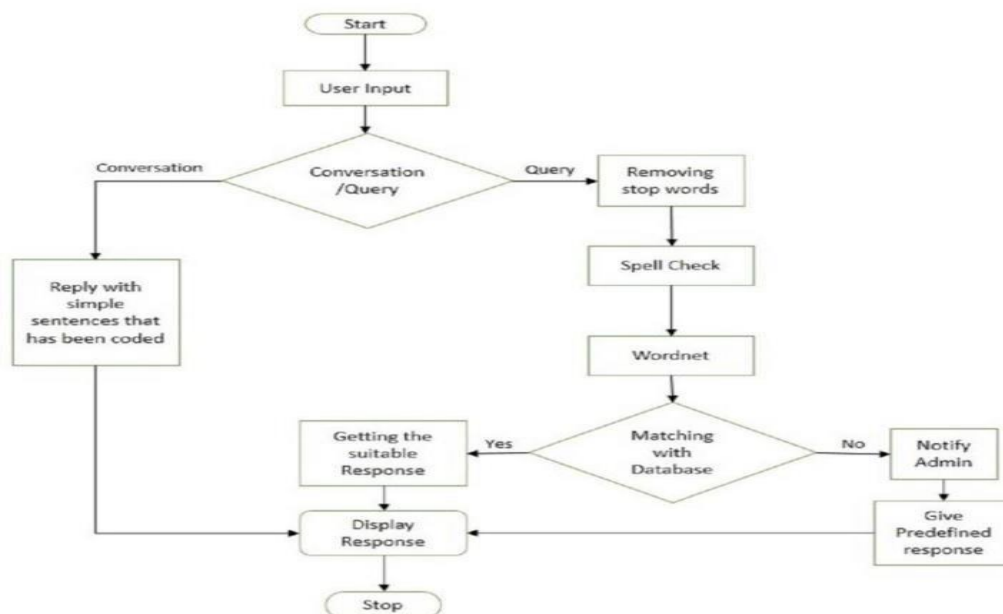
3. Artificial Neural Networks

Here, each sentence is broken down into individual words, and each word is then used as input for the neural networks.

The weighted connections are then calculated by different iterations through the training data thousands of times, each time improving the weights to make it accurate.



We will be using the ANN method for our Chatbot.



5. Architecture

Components of a chatbot

1. The user flow - Starting with intents

To generate a response, that chatbot has to understand what the user is trying to say i.e., it has to understand the user's intent.

2. Fetching a response

This information is used to:

- a) Respond to the user with a message defined by the rules set by the bot builder
- b) Retrieve data from your database
- c) Make an API call to get results matching intent

3. Backend Integration

Since chatbots rely on information and services exposed by other systems or applications through APIs, this module interacts with those applications or systems via APIs.

4. Question and Answer System

The Q&A system is responsible for answering or handling frequent customer queries. Developers can manually train the bot or use automation to respond to customer queries. The Q&A system automatically picks up the answers or solutions from the given database based on the customer intent.

Model of Chatbot used

Retrieval-based model

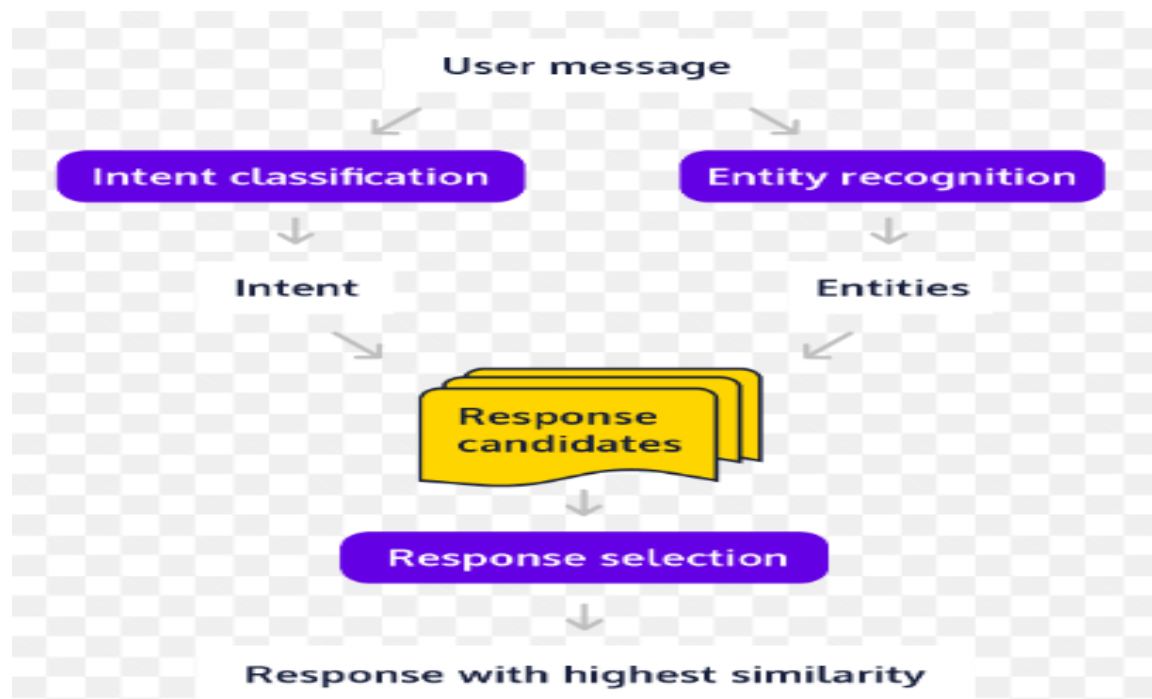
The chatbot uses the message and context of the conversation for selecting the best response from a predefined list of bot messages. The context can include the current position in the dialog tree, all previous messages in the conversation, and previously saved variables. They also provide more predictable results.

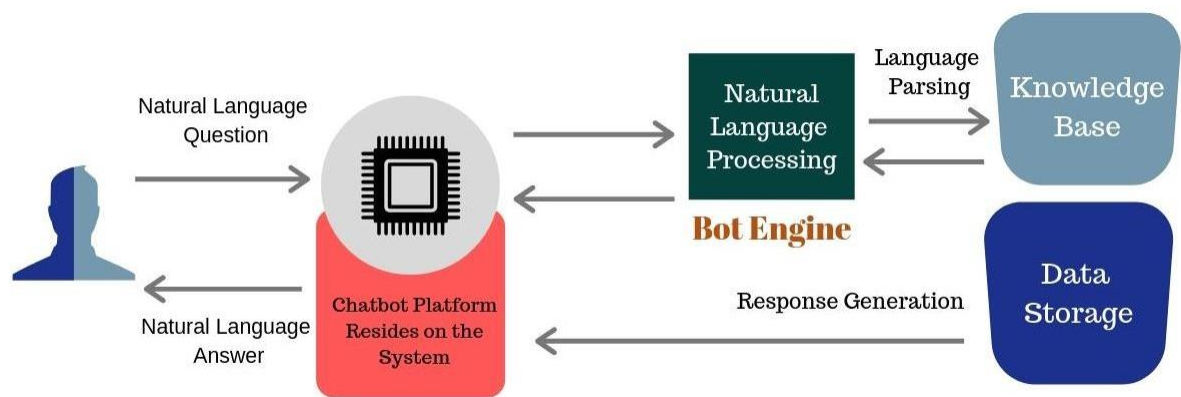
To achieve this model, we used,

Manual Training:

The process in which an expert creates FAQs (Frequently asked questions) and then maps them with relevant answers is known as manual training. This helps the bot identify important questions and answer them effectively.

Structure of Retrieval-based model





Dataset used:

Here are the datasets used:

1:

The dataset used in the first code is a custom JSON file named "intents.json". It contains a set of user intents and their corresponding patterns, tags, and responses. This dataset is used to train a neural network-based chatbot model.

2:

The second code does not use any dataset. It is a function that takes user input as text and uses the pre-trained chatbot model, tokenizer, and label encoder to generate a response.

3:

The third code sample also uses a custom JSON file named "voc.json". It contains a set of English words along with their frequency of occurrence in the English language. This dataset is used to train a neural network-based language model.

Tools used:

System Requirements

- Processor: 2 GHz or more
- Memory (RAM): 4 GB or above
- Hard Drive: 64 GB or more
- Sound card with speakers
- Microphone

Software Used

- OS: Windows 10, Ubuntu
- Anaconda
- Chrome, Firefox
- TensorFlow
- Keras
- Pickle
- Flask
- NLTK

Libraries

Nltk-- NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum.

Flask-- Flask is a web framework, it's a Python module that lets you develop web applications easily. It's has a small and easy-to-extend core: it's a microframework that doesn't include an ORM (Object Relational Manager) or such features

json5-- JSON5 is an extension to the popular JSON file format that aims to be easier to write and maintain *by hand* (e.g. for config files). It is *not intended* to be used for machine-to-machine communication.

Numpy-- NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

Tensorflow-- TensorFlow makes it easy for beginners and experts to create machine learning models for desktop, mobile, web, and cloud. See the sections below to get started.

Keras-- Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides.

Screenshot and Demo:

Code:

```

model_train.py > ...
1  import json
2  import numpy as np
3  import tensorflow as tf
4  from tensorflow import keras
5  from tensorflow.keras.models import Sequential
6  from tensorflow.keras.layers import Dense, Embedding, GlobalAveragePooling1D
7  from tensorflow.keras.preprocessing.text import Tokenizer
8  from tensorflow.keras.preprocessing.sequence import pad_sequences
9  from sklearn.preprocessing import LabelEncoder
10
11
12  with open('intents.json') as file:
13      data = json.load(file)
14
15  training_sentences = []
16  training_labels = []
17  labels = []
18  responses = []
19
20
21  for intent in data['intents']:
22      for pattern in intent['patterns']:
23          training_sentences.append(pattern)
24          training_labels.append(intent['tag'])
25          responses.append(intent['responses'])
26
27      if intent['tag'] not in labels:
28          labels.append(intent['tag'])
29
30  num_classes = len(labels)
31
32
33  lbl_encoder = LabelEncoder()
34  lbl_encoder.fit(training_labels)
35  training_labels = lbl_encoder.transform(training_labels)
36
37  vocab_size = 1000
38  embedding_dim = 16
39  max_len = 20
40  oov_token = "<OOV>"
41
42  tokenizer = Tokenizer(num_words=vocab_size, oov_token=oov_token)
43  tokenizer.fit_on_texts(training_sentences)
44  word_index = tokenizer.word_index
45  sequences = tokenizer.texts_to_sequences(training_sentences)
46  padded_sequences = pad_sequences(sequences, truncating='post', maxlen=max_len)
47
48  model = Sequential()
49  model.add(Embedding(vocab_size, embedding_dim, input_length=max_len))
50  model.add(GlobalAveragePooling1D())
51  model.add(Dense(16, activation='relu'))
52  model.add(Dense(16, activation='relu'))
53  model.add(Dense(num_classes, activation='softmax'))
54
55  model.compile(loss='sparse_categorical_crossentropy',
56              optimizer='adam', metrics=['accuracy'])
57
58  model.summary()
59  epochs = 500
60  history = model.fit(padded_sequences, np.array(training_labels), epochs=epochs)
61
62
63  # to save the trained model
64  model.save("chat_model")
65
66  import pickle
67
68  # to save the fitted tokenizer

```



```

main.py > ...
1  from spacy.lang.en import English
2  import numpy
3  from flask import Flask, render_template, request
4  import json
5  import pickle
6  import os
7  import time
8  import tensorflow as tf
9  from tensorflow.keras import layers, models, regularizers
10 from voc import voc
11 import random
12
13
14
15 app = Flask(__name__)
16
17
18
19 import json
20 import numpy as np
21 from tensorflow import keras
22 from sklearn.preprocessing import LabelEncoder
23
24 import colorama
25 colorama.init()
26 from colorama import Fore, Style, Back
27
28 import random
29 import pickle
30
31 with open("intents.json") as file:
32     data = json.load(file)
33
34
35 def chat(inp):
36     # load trained model
37     model = keras.models.load_model('chat_model')
38
39     # load tokenizer object
40     with open('tokenizer.pickle', 'rb') as handle:
41         tokenizer = pickle.load(handle)
42
43     # load label encoder object
44     with open('label_encoder.pickle', 'rb') as enc:
45         lbl_encoder = pickle.load(enc)
46
47     # parameters
48     max_len = 20
49
50     while True:
51         print(Fore.LIGHTBLUE_EX + "User: " + Style.RESET_ALL, end="")
52         # inp = input()
53         if inp.lower() == "quit":
54             break
55
56         result = model.predict(keras.preprocessing.sequence.pad_sequences(tokenizer.texts_to_sequences([inp]),
57                                                                           truncating='post', maxlen=max_len))
58         tag = lbl_encoder.inverse_transform([np.argmax(result)])
59         newResult = "MM"
60         for i in data['intents']:
61             if i['tag'] == tag:
62                 newResult = np.random.choice(i['responses'])
63         return newResult
64         # print(Fore.GREEN + "ChatBot:" + Style.RESET_ALL, random.choice(responses))
65
66 print(Fore.YELLOW + "Start messaging with the bot (type quit to stop)!" + Style.RESET_ALL)
67
68 @app.route("/")

```

This is a Python script that trains a chatbot model using a neural network with the TensorFlow library. Here's a brief overview of what each section of the script does:

1. Import necessary libraries: The script starts by importing the ``json`` library for reading the training data from a JSON file, as well as other libraries including NumPy, TensorFlow, and scikit-learn.
2. Load training data: The script loads the training data from a JSON file named ``intents.json``. The data contains different intents (such as "greeting", "goodbye", "thanks", etc.) and example patterns that correspond to each intent. The patterns are the sentences or phrases that users might type in to trigger each intent.
3. Preprocess training data: The script preprocesses the training data by extracting the training sentences and labels from the JSON data, converting the labels to numerical values using scikit-learn's ``LabelEncoder``, tokenizing the sentences using TensorFlow's ``Tokenizer``, and padding the sequences to a fixed length using TensorFlow's ``pad_sequences``.
4. Define the neural network model: The script defines a neural network model using the Keras API from TensorFlow. The model consists of an embedding layer, a global average pooling layer, two dense layers with ReLU activation functions, and a final dense layer with a softmax activation function.
5. Compile the model: The script compiles the model by specifying the loss function, optimizer, and evaluation metric to use during training.
6. Train the model: The script trains the model using the ``fit()`` method from Keras. The training data is passed in along with the number of epochs to train for.
7. Save the trained model: The script saves the trained model to a file named "chat_model" using Keras' ``save()`` method.
8. Save the fitted tokenizer: The script saves the fitted tokenizer to a file named "tokenizer.pickle" using Python's ``pickle`` library.
9. Save the fitted label encoder: The script saves the fitted label encoder to a file named "label_encoder.pickle" using ``pickle``.

NLP concepts used:

1. Tokenization: The ``Tokenizer`` class from TensorFlow's ``preprocessing.text`` module is used to tokenize the training sentences. This involves splitting each sentence into individual words or tokens, and creating a vocabulary of unique tokens.
2. Padding: The ``pad_sequences`` function from TensorFlow's ``preprocessing.sequence`` module is used to pad the tokenized sequences to a fixed length. This ensures that all input sequences have the same length, which is required for training the neural network.
3. Word embeddings: The ``Embedding`` layer in the neural network model is used to create word embeddings for each token in the input sequences. Word embeddings are dense vectors that represent the meaning of each word in the vocabulary.
4. Label encoding: The ``LabelEncoder`` class from scikit-learn is used to convert the string labels into numerical values. This is necessary because neural networks require numerical input and output values.
5. Sparse categorical cross-entropy loss: The ``sparse_categorical_crossentropy`` loss function is used to compute the difference between the predicted and actual class labels during training. This is a common loss function used in multi-class classification problems.

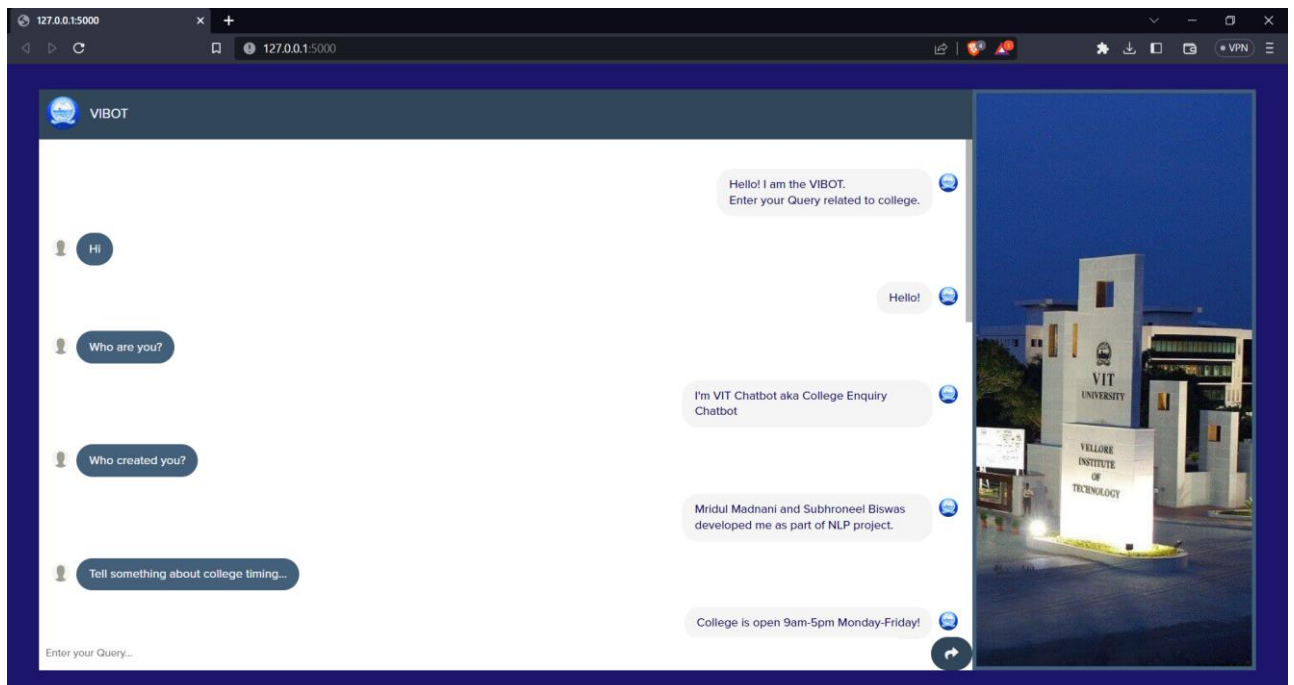
Output Screenshot:

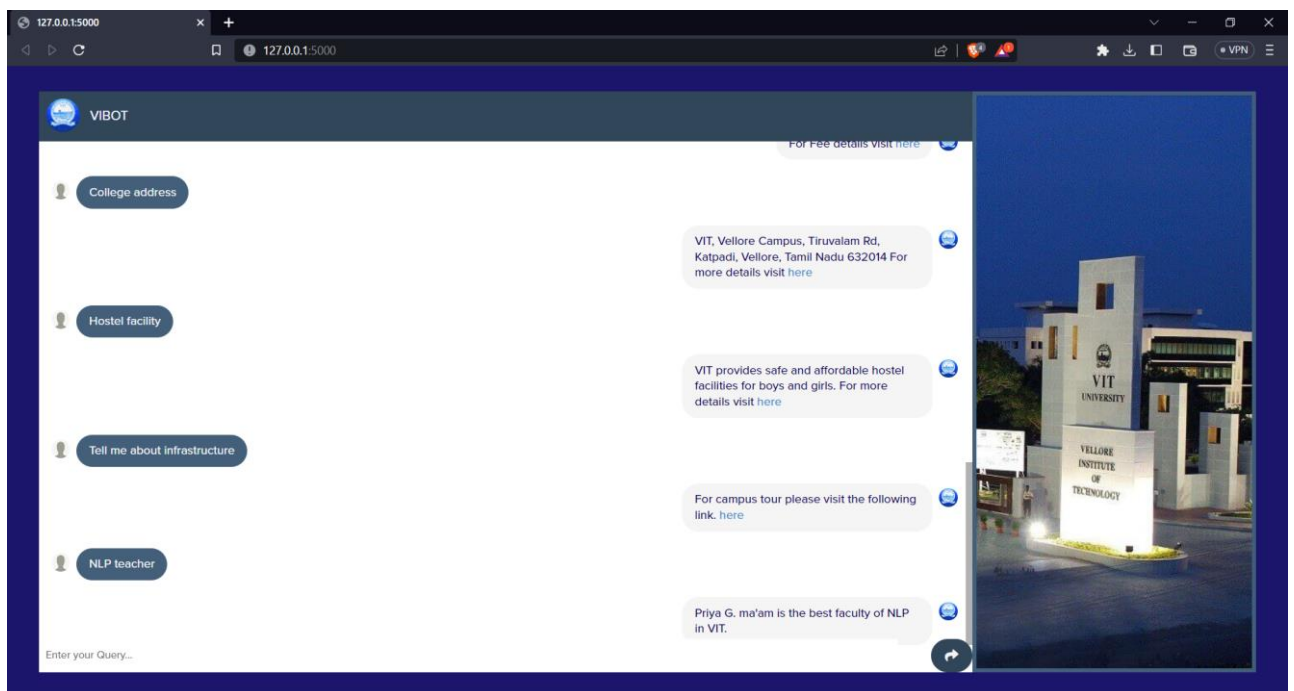
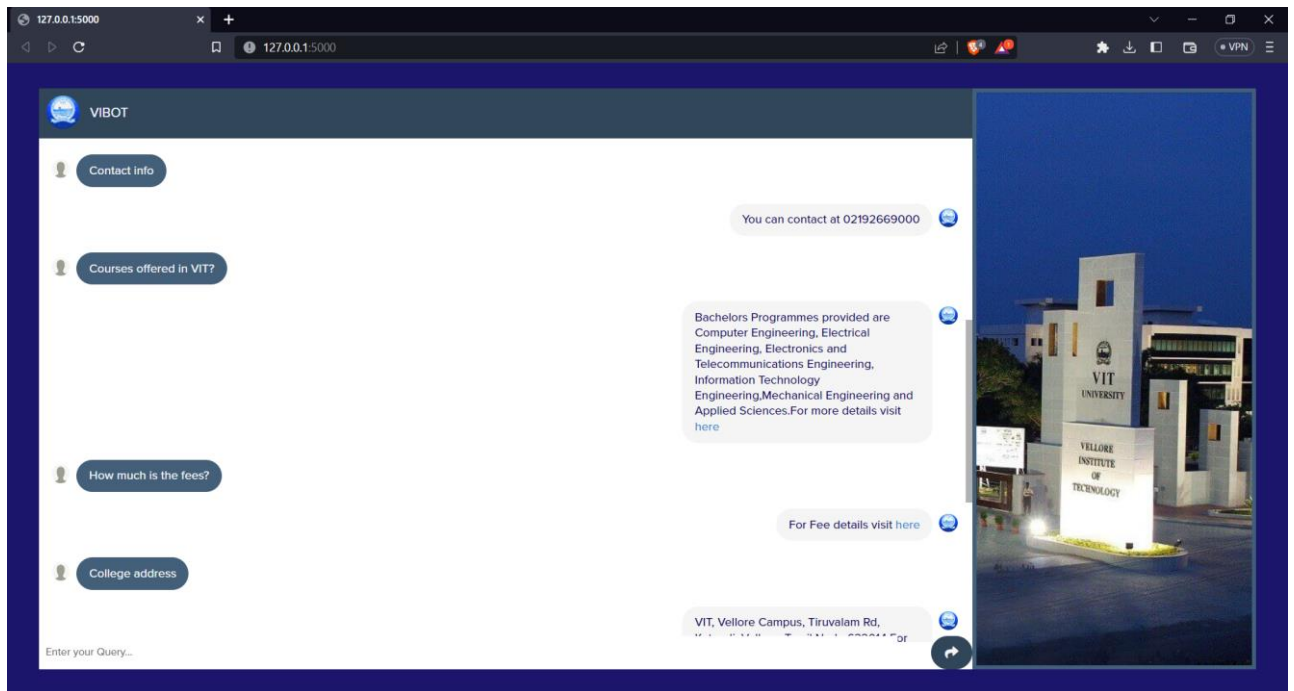
```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL
Epoch 4/4/500
15/15 [=====] - 0s 2ms/step - loss: 0.0103 - accuracy: 0.9978
Epoch 475/500
15/15 [=====] - 0s 3ms/step - loss: 0.0108 - accuracy: 0.9978
Epoch 476/500
15/15 [=====] - 0s 2ms/step - loss: 0.0087 - accuracy: 0.9978
Epoch 477/500
15/15 [=====] - 0s 3ms/step - loss: 0.0098 - accuracy: 0.9978
Epoch 478/500
15/15 [=====] - 0s 1ms/step - loss: 0.0091 - accuracy: 0.9978
Epoch 479/500
15/15 [=====] - 0s 2ms/step - loss: 0.0091 - accuracy: 0.9978
Epoch 480/500
15/15 [=====] - 0s 2ms/step - loss: 0.0086 - accuracy: 0.9978
Epoch 481/500
15/15 [=====] - 0s 3ms/step - loss: 0.0087 - accuracy: 0.9978
Epoch 482/500
15/15 [=====] - 0s 2ms/step - loss: 0.0094 - accuracy: 0.9956
Epoch 483/500
15/15 [=====] - 0s 2ms/step - loss: 0.0084 - accuracy: 0.9956
Epoch 484/500
15/15 [=====] - 0s 3ms/step - loss: 0.0088 - accuracy: 0.9956
Epoch 485/500
15/15 [=====] - 0s 2ms/step - loss: 0.0096 - accuracy: 0.9956
Epoch 486/500
15/15 [=====] - 0s 2ms/step - loss: 0.0088 - accuracy: 0.9978
Epoch 487/500
15/15 [=====] - 0s 2ms/step - loss: 0.0088 - accuracy: 0.9978
Epoch 488/500
15/15 [=====] - 0s 2ms/step - loss: 0.0139 - accuracy: 0.9978
Epoch 489/500
15/15 [=====] - 0s 3ms/step - loss: 0.0095 - accuracy: 0.9978
Epoch 490/500
15/15 [=====] - 0s 2ms/step - loss: 0.0095 - accuracy: 0.9978
Epoch 491/500
15/15 [=====] - 0s 2ms/step - loss: 0.0081 - accuracy: 0.9978
Epoch 492/500
15/15 [=====] - 0s 2ms/step - loss: 0.0089 - accuracy: 0.9978
Epoch 493/500
15/15 [=====] - 0s 2ms/step - loss: 0.0092 - accuracy: 0.9978
Epoch 494/500
15/15 [=====] - 0s 2ms/step - loss: 0.0119 - accuracy: 0.9978
Epoch 495/500
Epoch 496/500
15/15 [=====] - 0s 2ms/step - loss: 0.0093 - accuracy: 0.9956
Epoch 497/500
15/15 [=====] - 0s 2ms/step - loss: 0.0095 - accuracy: 0.9978
Epoch 498/500
15/15 [=====] - 0s 2ms/step - loss: 0.0102 - accuracy: 0.9978
Epoch 499/500
15/15 [=====] - 0s 2ms/step - loss: 0.0106 - accuracy: 0.9956
Epoch 500/500
```

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL
Epoch 500/500
15/15 [=====] - 0s 2ms/step - loss: 0.0083 - accuracy: 0.9978
WARNING:absl:Found untraced functions such as _update_step_xla while saving (showing 1 of 1). These functions will not be directly callable after loading.
PS C:\Users\digit\Desktop\Desktop\VIT\6. Winter Semester 2022-23\NLP\Project\VIBOT> & C:\Users\digit\anaconda3\python.exe "c:/Users/digit/Desktop/Desktop/VIT/6. Winter Semester 2022-23/NLP/Project/VIBOT/main.py"
Start messaging with the bot (type quit to stop)!
* Serving Flask app 'main'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [12/Apr/2023 00:38:21] "GET / HTTP/1.1" 200 -
1/1 [=====] - 0s 75ms/step
127.0.0.1 - - [12/Apr/2023 00:38:29] "GET /get?msg=Hi HTTP/1.1" 200 -
1/1 [=====] - 0s 60ms/step
127.0.0.1 - - [12/Apr/2023 00:38:38] "GET /get?msg=Namaste HTTP/1.1" 200 -
1/1 [=====] - 0s 44ms/step
127.0.0.1 - - [12/Apr/2023 00:38:42] "GET /get?msg=Salaam HTTP/1.1" 200 -
1/1 [=====] - 0s 49ms/step
127.0.0.1 - - [12/Apr/2023 00:38:47] "GET /get?msg=Vanakkam HTTP/1.1" 200 -
127.0.0.1 - - [12/Apr/2023 00:50:22] "GET / HTTP/1.1" 200 -
WARNING:tensorflow:5 out of the last 5 calls to <function Model.make_predict_function.<locals>.predict_function at 0x00000249AF5D6830> triggered tf.function retracing
g. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shape
s, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True optio
n that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_d
ocs/python/tf/function for more details.
1/1 [=====] - 0s 179ms/step
127.0.0.1 - - [12/Apr/2023 00:50:32] "GET /get?msg=Hi HTTP/1.1" 200 -
WARNING:tensorflow:6 out of the last 6 calls to <function Model.make_predict_function.<locals>.predict_function at 0x00000249AF655480> triggered tf.function retracin
g. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shape
s, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True optio
n that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_d
ocs/python/tf/function for more details.
1/1 [=====] - 0s 50ms/step
127.0.0.1 - - [12/Apr/2023 00:51:52] "GET /get?msg=Who+are+you%3F HTTP/1.1" 200 -
1/1 [=====] - 0s 49ms/step
127.0.0.1 - - [12/Apr/2023 00:52:42] "GET /get?msg=Who+created+you%3F HTTP/1.1" 200 -
1/1 [=====] - 0s 50ms/step
127.0.0.1 - - [12/Apr/2023 00:53:19] "GET /get?msg=Tell+something+about+college+timing... HTTP/1.1" 200 -
1/1 [=====] - 0s 46ms/step
127.0.0.1 - - [12/Apr/2023 00:53:44] "GET /get?msg=Contact+info HTTP/1.1" 200 -
1/1 [=====] - 0s 61ms/step
127.0.0.1 - - [12/Apr/2023 00:54:21] "GET /get?msg=Courses+offered+in+VIT%3F HTTP/1.1" 200 -
1/1 [=====] - 0s 49ms/step
127.0.0.1 - - [12/Apr/2023 00:55:20] "GET /get?msg=How+much+is+the+fees%3F HTTP/1.1" 200 -
1/1 [=====] - 0s 46ms/step
127.0.0.1 - - [12/Apr/2023 00:55:46] "GET /get?msg=College+address HTTP/1.1" 200 -
1/1 [=====] - 0s 63ms/step
127.0.0.1 - - [12/Apr/2023 00:56:30] "GET /get?msg=Hostel+facility HTTP/1.1" 200 -
1/1 [=====] - 0s 49ms/step
127.0.0.1 - - [12/Apr/2023 00:57:02] "GET /get?msg=Tell+me+about+infrastructure HTTP/1.1" 200 -
1/1 [=====] - 0s 44ms/step

```





Results and discussion:

The results and outcomes of the above-defined project, which is a VIT chatbot, can be:

1. **Improved student experience:** The chatbot can provide quick and accurate responses to students' queries, enhancing their experience with the college. It can provide personalized guidance to students in real-time, which can help improve student engagement and satisfaction.
2. **Increased efficiency:** The chatbot can handle multiple queries simultaneously, freeing up the college's staff and resources. The chatbot can be available 24/7, providing students with assistance whenever they need it.
3. **Better data analysis:** The chatbot can collect data on student queries and interactions, providing valuable insights into students' needs and interests. This data can help the college improve its services and offerings to better meet the needs of its students.
4. **Improved outreach:** The chatbot can be integrated with the college's social media channels, allowing it to reach a wider audience of potential students. It can provide information on admissions, courses, and campus life, helping to attract new students to the college.
5. **Cost-effective solution:** The chatbot can provide cost-effective assistance to students, reducing the need for hiring additional staff to handle student queries. It can also reduce the need for physical resources like brochures and information packets.

In terms of routine experimental results, the performance of the chatbot can be measured based on metrics such as accuracy, response time, and customer satisfaction. These metrics can be collected and analyzed over time to make improvements to the chatbot's functionality and performance.

Conclusion And Future Work:

The project demonstrates the development of a specific college chatbot using natural language processing techniques. The project uses a neural network-based approach to train a chatbot model using the given dataset of intents and responses.

The chatbot can understand user input and provide relevant responses based on the intent of the input. The chatbot model can be integrated into various college websites, portals, and other related platforms to provide automated assistance to students, faculty, and other users.

In conclusion, the project demonstrates the potential of chatbots to improve user engagement and experience by providing efficient and personalized assistance. The project can be further improved by incorporating additional features such as voice-based inputs and outputs, sentiment analysis, and improved handling of complex user queries.

Future work can also focus on expanding the dataset to cover a wider range of intents and responses, and incorporating multi-lingual support to make the chatbot accessible to a wider audience. The chatbot can also be integrated with other systems such as course management systems, student information systems, and other relevant platforms to provide a more comprehensive solution. Overall, the project has the potential to transform the way colleges interact with their students and other stakeholders, making education more accessible and convenient for everyone involved.

In Future instead of an AI or ML based bot, other algorithms can be implemented. We can include voice-based queries. The users will have to give voice input and the system will give the text output.

Also, after successful execution of chatbot in college domain, we can implement it in other domains like medical, forensic, sports, etc. It will be beneficial in all the fields as without spending much time, we are accessing the relevant information and that too without any sorting.

References

- 1) 1. A., Sameera and John, Dr. (2015) 'Survey on Chatbot Design Techniques in Speech Conversation Systems', International Journal of Advanced Computer Science and Applications, 6, doi: 10.14569/IJACSA.2015.060712.
- 2) 2. Dibitonto, Massimiliano, Leszczynska, Katarzyna, Tazzi, Federica and Medaglia, Carlo (2018) 'Chatbot in a Campus Environment: Design of LiSA, a Virtual Assistant to Help Students in Their University Life', doi: 10.1007/978-3-319-91250-9_9.
- 3) 3. Reynald, D. J. (2017) 'NATURAL LANGUAGE PROCESSING USING PYTHON', International Journal of Scientific & Engineering Research, 8(5), <http://www.ijser.org>.
- 4) 4. Shakhovska, N., Basystiuk, O. and Shakhovska, K. (2019) 'Development of the Speech-to-Text Chatbot Interface Based on Google API', in MoMLeT, pp. 212-221.
- 5) 5. Ayanouz, S., Abdelhakim, B. A. and Benhmed, M. (2020, March) 'A smart chatbot architecture based NLP and machine learning for health care assistance', in Proceedings of the 3rd International Conference on Networking, Information Systems & Security, pp. 1-6.
- 6) 6. Alemi, A. A. and Ginsparg, P. (2015) 'Text segmentation based on semantic word embeddings', arXiv preprint arXiv:1503.05543.
- 7) 7. Kulkarni, C. S., Bhavsar, A. U., Pingale, S. R. and Kumbhar, S. S. (2017) 'BANK CHAT BOT—an intelligent assistant system using NLP and machine learning', International Research Journal of Engineering and Technology, 4(5), pp. 2374-2377.
- 8) 8. Pradeep, J., Vijayakumar, K. and Harikrishnan, M. (2022) 'Voice Recognition Using Natural Language Processing', in Big data management in Sensing, pp. 15-23. River Publishers.
- 9) 9. Yuhua Li, David McLean, Zuhair A. Bandar, James D. O'Shea, Keeley Crockett (2006) 'Sentence Similarity Based on Semantic Nets and Corpus Statistics', IEEE Transactions on Knowledge and Data Engineering, 18(8).
- 10) 10. Emanuela Haller, Traian Rebedea (2013) 'Designing a Chatbot that Simulates an Historical Figure', IEEE Conference Publications.

VIT CHATBOT

Mridul Madnani

20BDS0191

Vellore Institute of Technology, Vellore

mridul.madnani2020@vitstudent.ac.in

Subhrooneel Biswas

20BDS0266

Vellore Institute of Technology, Vellore

subhrooneel.biswas2020@vitstudent.ac.in

I. INTRODUCTION

In recent years, we've become familiar with chatbots and how beneficial they can be for business owners, employees, and customers alike. Despite what we're used to and how their actions are fairly limited to scripted conversations and responses, the future of chatbots is life-changing, to say the least. The standard usage might not require more than quick answers and simple replies, but it's important to know just how much chatbots are evolving and how Natural Language Processing (NLP) can improve their abilities. This function holds plenty of rewards, really putting the 'chat' in the chatbot. This seemingly complex process can be identified as one which allows computers to derive meaning from text inputs. Put simply, NLP is an applied artificial intelligence (AI) program that helps the chatbot analyse and understand the natural human language communicated with the customers. Chatbots can understand the intent of the conversation rather than just use the information to communicate and respond to queries. Business owners are starting to feed their chatbots with actions to "help" them become more humanized and personal in their chats. It's incredible just how intelligent chatbots can be if you take the time to feed them the information they need to evolve and make a difference in the business. This intent-driven function will be able to bridge the gap between customers and businesses, making sure that the chatbot is something customers want to speak to when communicating with the business. We have built a chatbot that is able to answer any queries regarding VIT university. All the information on faculty, courses, transportation, events, etc. can be easily accessed through our chatbot. The final scope of this work is to design a chatbot that could help students in their campus life.

II. LITERATURE SURVEY

Authors and Year (Reference)	Title (Study)	Concept /Theoretical Model/ Framework	Methodology Used /Implementation	Dataset Details/ Analysis	Relevant Findings	Limitations/ Future Research/Gap Identified
Sameera and John (2015)	Survey on Chatbot Design Techniques in Speech Conversation Systems	Design Techniques for Chatbots	Survey and Literature Review	Theoretical aspect for VOI chatbots	Various chatbot design techniques, strengths and weaknesses of each technique, and areas for improvement	Lack of empirical studies, limited focus on speech-based chatbots, need for user evaluation and testing
Dibbitonto et al. (2018)	Chatbot in a Campus Environment: Design of LiSA, a Virtual Assistant to Help Students in Their University Life	LiSA chatbot framework for campus environments	Design and implementation of LiSA chatbot framework	Analysis of theoretical aspect for LiSA	LiSA chatbot framework can provide personalized assistance to students, improve student engagement and satisfaction, and reduce administrative workload for staff	Limited evaluation of LiSA framework, need for further testing and improvement

Reynald (2017)	Natural Language Processing using Python	Natural Language Processing concepts and techniques	Python programming and implementation of NLP algorithms	Theoretical aspects of Python with NLP	Overview of NLP concepts and techniques, and practical implementation on examples using Python	Limited focus on chatbot implementation, need for more in-depth discussion and analysis
Shakhovska et al. (2019)	Development of the Speech-to-Text Chatbot Interface Based on Google API	Speech-to-Text Chatbot Interface using Google API	Design and implementation of chatbot interface using Google API	Analysis of theoretical aspect for Assistants	Speech-to-Text chatbot interface can improve accessibility and ease of use for users, and reduce costs for developers	Limited evaluation of interface, need for more user testing and feedback
Ayanouz et al. (2020)	A Smart Chatbot Architecture based NLP and Machine Learning for Health Care Assistance	Chatbot architecture for healthcare assistance	Design and implementation of chatbot architecture using NLP and ML	Medical dataset with patient symptoms and diseases	Chatbot can provide accurate diagnosis and treatment recommendations, and reduce healthcare costs and workload	Limited evaluation on real-world patient data, need for further testing and refinement

Reynald (2017)	Natural Language Processing using Python	Natural Language Processing concepts and techniques	Python programming and implementation of NLP algorithms	Theoretical aspects of Python with NLP	Overview of NLP concepts and techniques, and practical implementation on examples using Python	Limited focus on chatbot implementation, need for more in-depth discussion and analysis
Shakhovska et al. (2019)	Development of the Speech-to-Text Chatbot Interface Based on Google API	Speech-to-Text Chatbot Interface using Google API	Design and implementation of chatbot interface using Google API	Analysis of theoretical aspect for Assistants	Speech-to-Text chatbot interface can improve accessibility and ease of use for users, and reduce costs for developers	Limited evaluation of interface, need for more user testing and feedback
Ayanouz et al. (2020)	A Smart Chatbot Architecture based NLP and Machine Learning for Health Care Assistance	Chatbot architecture for healthcare assistance	Design and implementation of chatbot architecture using NLP and ML	Medical dataset with patient symptoms and diseases	Chatbot can provide accurate diagnosis and treatment recommendations, and reduce healthcare costs and workload	Limited evaluation on real-world patient data, need for further testing and refinement

III. OBJECTIVE

The objective of creating a specific college chatbot using the above codes could be to provide a virtual assistant to students for answering their queries related to the college such as admission process, courses offered, fees structure, facilities available, important dates, etc. This chatbot can also provide personalized guidance and support to students by analyzing their academic performance, interests, and preferences, and recommending suitable courses, programs, or activities. The chatbot can also assist the college staff by answering frequently asked questions and providing timely updates to students and parents. Overall, the objective is to create a user-friendly and efficient communication channel between students and the college administration.

NLP bots generate responses based on user inputs. At times, constraining user input can be a great way to focus and speed up query resolution. So, when logical, falling back upon rich elements such as buttons, carousels or quick replies won't make the bot seem any less intelligent. To the contrary, besides the speed, rich controls also help to reduce users' cognitive load. Hence, they don't need to wonder about what is the right thing to say or ask. When in doubt, always opt for simplicity. The lack of a conversation ender can easily become an issue and you would be surprised how many NLP chatbots don't have one. If the user isn't sure whether or not the conversation has ended the bot might end up looking stupid or it will force you to work on further intents that would have otherwise been unnecessary. But the rule and choice-based bots don't have this issue. By considering the above points, we have created an NLP chatbot that is capable of answering most of the possible questions about VIT. We thought of all the possible scenarios in which our chatbot will be useful and designed it in an impeccable way.

IV. INNOVATION

The project differs from others by incorporating specific college-related information and ensuring that the responses provided by the chatbot are accurate and up-to-date. It can also be designed to handle user inputs in natural language, making the conversation with the chatbot more intuitive and user-friendly. Additionally, the chatbot can be integrated with the college's website and social media channels to provide students and prospective students with easy access to information.

The routine experimental results of such a project can include metrics such as user satisfaction, response accuracy, and response time. The chatbot's performance can be evaluated

by comparing its responses to the queries with the actual answers provided by the college. The project's success can be measured by the level of user engagement and feedback, as well as the reduction in the workload on college staff in handling routine queries.

The project of building a VIT chatbot using NLP and python code as a base, the innovation component is in the form of customizing the chatbot to cater to the specific needs and requirements of the college i.e. VIT. For example, the chatbot can be designed to answer queries related to admission procedures, course details, campus facilities, student life, and events, among other things.

Some possible innovations for this project include:

1. *Incorporated sentiment analysis*: The chatbot is trained to detect the sentiment of the user's input and respond accordingly. This can be useful in situations where the user is expressing frustration or dissatisfaction with the college.
2. *Implemented multi-lingual support*: The chatbot is trained to recognize and respond to user input in different languages. This can be useful in colleges with a diverse student population.
3. *Added personalized recommendations*: The chatbot is trained to provide personalized recommendations based on the user's interests and preferences, such as clubs or activities to join, classes to take, or resources to access.

Overall, these innovations can enhance the user experience and make the chatbot more useful and engaging for students.

v. METHODOLOGY

Chatbots work by using three classification methods:

1. Pattern matching

Pattern matching to analyse the text and produce a suitable response. The standard structure of these patterns is Artificial Intelligence Markup Language.

For example:

<pattern> Who is the developer? </pattern>

<template> Mridul Madnani and Subhroneel Biswas developed me as part of NLP project. </template>

Chatbot knows the answer because his or her name is part of an associated pattern. But for more advanced information,

which is beyond the related pattern, the chatbot will need to use algorithms.

2. Algorithms

Algorithms reduce the number of classifiers and create a more manageable structure. In the following example, each class is assigned a score.

Term: "Good" (class: Greetings)

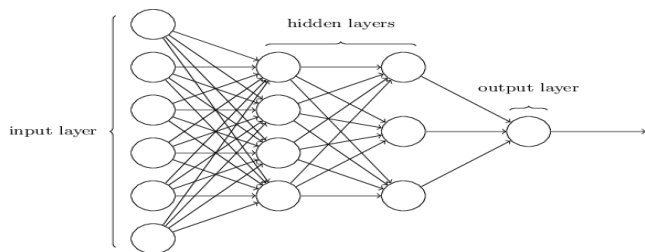
Term: "morning" (class: Greetings) Classification: Greetings (score=2)

With the help of an equation, word matches are found for the given sentence, and this identifies the class with the highest match.

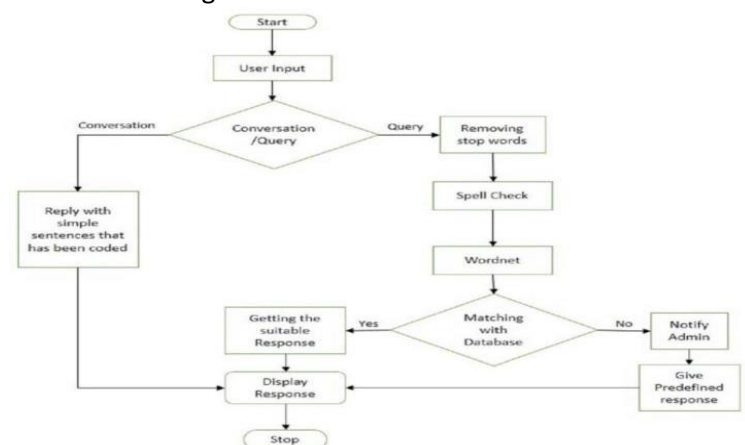
3. Artificial Neural Networks

Here, each sentence is broken down into individual words, and each word is then used as input for the neural networks.

The weighted connections are then calculated by different iterations through the training data thousands of times, each time improving the weights to make it accurate.



We will be using the ANN method for our Chatbot.



Architecture

Components of a chatbot

1. The user flow - Starting with intents

To generate a response, that chatbot has to understand what the user is trying to say i.e., it has to understand the user's intent.

2. Fetching a response

This information is used to:

- Respond to the user with a message defined by the rules set by the bot builder
- Retrieve data from your database
- Make an API call to get results matching intent

3. Backend Integration

Since chatbots rely on information and services exposed by other systems or applications through APIs, this module interacts with those applications or systems via APIs.

4. Question and Answer System

The Q&A system is responsible for answering or handling frequent customer queries. Developers can manually train the bot or use automation to respond to customer queries. The Q&A system automatically picks up the answers or solutions from the given database based on the customer intent.

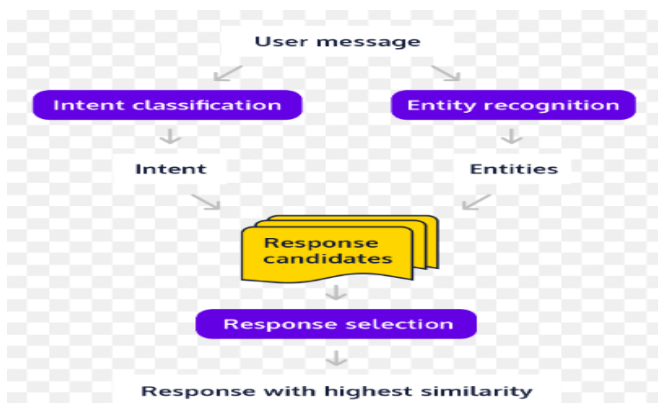
Model of Chatbot used

Retrieval-based model

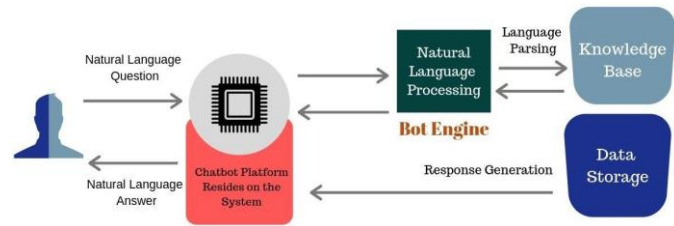
The chatbot uses the message and context of the conversation for selecting the best response from a predefined list of bot messages. The context can include the current position in the dialog tree, all previous messages in the conversation, and previously saved variables. They also provide more predictable results.

To achieve this model, we used, Manual Training:

The process in which an expert creates FAQs (Frequently asked questions) and then maps them with relevant answers is known as manual training. This helps the bot identify important questions and answer them effectively.



Structure of Retrieval-based model



VI. DATASET

Here are the datasets used:

The dataset used in the first code is a custom JSON file named "intents.json". It contains a set of user intents and their corresponding patterns, tags, and responses. This dataset is used to train a neural network-based chatbot model.

The second code does not use any dataset. It is a function that takes user input as text and uses the pre-trained chatbot model, tokenizer, and label encoder to generate a response.

The third code sample also uses a custom JSON file named "voc.json". It contains a set of English words along with their frequency of occurrence in the English language. This dataset is used to train a neural network-based language model.

VII. TOOLS

System Requirements

- Processor: 2 GHz or more
- Memory (RAM): 4 GB or above
- Hard Drive: 64 GB or more
- Sound card with speakers
- Microphone

Software Used

- OS: Windows 10, Ubuntu
- Anaconda
- Chrome, Firefox
- TensorFlow
- Keras
- Pickle
- Flask
- NLTK

Libraries

Nltk-- NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum.

Flask-- Flask is a web framework, it's a Python module that lets you develop web applications easily. It's has a small and easy-to-extend core: it's a microframework that doesn't include an ORM (Object Relational Manager) or such features

json5-- JSON5 is an extension to the popular [JSON](#) file format that aims to be easier to write and maintain *by hand* (e.g. for config files). It is *not intended* to be used for machine-to-machine communication.

Numpy-- NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

Tensorflow-- TensorFlow makes it easy for beginners and experts to create machine learning models for desktop, mobile, web, and cloud. See the sections below to get started.

Keras-- Keras is an API designed for human beings, not machines. Keras follows best practices for reducing cognitive load: it offers consistent & simple APIs, it minimizes the number of user actions required for common use cases, and it provides clear & actionable error messages. It also has extensive documentation and developer guides.

VIII. RESULTS

The results and outcomes of the above-defined project, which is a VIT chatbot, can be:

1. Improved student experience: The chatbot can provide quick and accurate responses to students' queries, enhancing their experience with the college.

It can provide personalized guidance to students in real-time, which can help improve student engagement and satisfaction.

2. Increased efficiency: The chatbot can handle multiple queries simultaneously, freeing up the college's staff and resources. The chatbot can be available 24/7, providing students with assistance whenever they need it.
3. Better data analysis: The chatbot can collect data on student queries and interactions, providing valuable insights into students' needs and interests. This data can help the college improve its services and offerings to better meet the needs of its students.
4. Improved outreach: The chatbot can be integrated with the college's social media channels, allowing it to reach a wider audience of potential students. It can provide information on admissions, courses, and campus life, helping to attract new students to the college.
5. Cost-effective solution: The chatbot can provide cost-effective assistance to students, reducing the need for hiring additional staff to handle student queries. It can also reduce the need for physical resources like brochures and information packets.

In terms of routine experimental results, the performance of the chatbot can be measured based on metrics such as accuracy, response time, and customer satisfaction. These metrics can be collected and analyzed over time to make improvements to the chatbot's functionality and performance.

IX. CONCLUSION

The project demonstrates the development of a specific college chatbot using natural language processing techniques. The project uses a neural network-based approach to train a chatbot model using the given dataset of intents and responses.

The chatbot can understand user input and provide relevant responses based on the intent of the input. The chatbot model can be integrated into various college websites, portals, and other related platforms to provide automated assistance to students, faculty, and other users.

In conclusion, the project demonstrates the potential of chatbots to improve user engagement and experience by providing efficient and personalized assistance. The project can be further improved by incorporating additional features such as voice-based inputs and outputs, sentiment analysis, and improved handling of complex user queries.

Future work can also focus on expanding the dataset to cover a wider range of intents and responses, and incorporating multi-lingual support to make the chatbot accessible to a wider audience. The chatbot can also be integrated with other systems such as course management systems, student information systems, and other relevant platforms to provide a more comprehensive solution. Overall, the project has the potential to transform the way colleges interact with their students and other stakeholders, making education more accessible and convenient for everyone involved.

In Future instead of an AI or ML based bot, other algorithms can be implemented. We can include voice-based queries. The users will have to give voice input and the system will give the text output.

Also, after successful execution of chatbot in college domain, we can implement it in other domains like medical, forensic, sports, etc. It will be beneficial in all the fields as without spending much time, we are accessing the relevant information and that too without any sorting.

X. REFERENCES

- 1) 1. A., Sameera and John, Dr. (2015) 'Survey on Chatbot Design Techniques in Speech Conversation Systems', International Journal of Advanced Computer Science and Applications, 6, doi: 10.14569/IJACSA.2015.060712.
- 2) 2. Dibitonto, Massimiliano, Leszczynska, Katarzyna, Tazzi, Federica and Medaglia, Carlo (2018) 'Chatbot in a Campus Environment: Design of LiSA, a Virtual Assistant to Help Students in Their University Life', doi: 10.1007/978-3-319-91250-9_9.
- 3) 3. Reynald, D. J. (2017) 'NATURAL LANGUAGE PROCESSING USING PYTHON', International Journal of Scientific & Engineering Research, 8(5), <http://www.ijser.org>.
- 4) 4. Shakhovska, N., Basystiuk, O. and Shakhovska, K. (2019) 'Development of the Speech-to-Text Chatbot Interface Based on Google API', in MoMLet, pp. 212-221.
- 5) 5. Ayanouz, S., Abdelhakim, B. A. and Benhmed, M. (2020, March) 'A smart chatbot architecture based NLP and machine learning for health care assistance', in Proceedings of the 3rd International Conference on Networking, Information Systems & Security, pp. 1-6.
- 6) 6. Alemi, A. A. and Ginsparg, P. (2015) 'Text segmentation based on semantic word embeddings', arXiv preprint arXiv:1503.05543.
- 7) 7. Kulkarni, C. S., Bhavsar, A. U., Pingale, S. R. and Kumbhar, S. S. (2017) 'BANK CHAT BOT—an intelligent assistant system using NLP and machine learning', International Research Journal of Engineering and Technology, 4(5), pp. 2374-2377.
- 8) 8. Pradeep, J., Vijayakumar, K. and Hari Krishnan, M. (2022) 'Voice Recognition Using Natural Language Processing', in Big data management in Sensing, pp. 15-23. River Publishers.
- 9) 9. Yuhua Li, David McLean, Zuhair A. Bandar, James D. O'Shea, Keeley Crockett (2006) 'Sentence Similarity Based on Semantic Nets and Corpus Statistics', IEEE Transactions on Knowledge and Data Engineering, 18(8).
- 10) 10. Emanuela Haller, Traian Rebedea (2013) 'Designing a Chat-bot that Simulates an Historical Figure', IEEE Conference Publications.

A Framework for an Adaptive Intrusion Detection System using Bayesian Network [1] proposed by Farah Jemli, Montaceur and Mohamad Ben through which they developed a Bayesian Network to identify known attack signatures. The goal of the Bayesian Network is to identify known attack signatures utilising a variety of attributes of modifications made by various packets, setting changes, and permissions obtained. Bayesian networks can recognise both typical and unusual attacks by continuously learning from audit data as well as historical data. One of the main drawbacks is that intrusion attacks evolve over time and become more challenging for intrusion detection systems utilising Bayesian networks to identify. Using probabilistic networks as an alternative to Bayesian networks can qualitatively depict the evaluation of intrusion risk.

A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection [2] by Anna L. Buczak and Erhan Guven focuses on cyber analytics, which makes use of data mining and machine learning. The complexity and difficulties that many machine learning or data mining approaches in cyber security face are addressed. Artificial Neural Networks, Association and Fuzzy Association Rules, Bayesian Network, Clustering, Decision Trees, Ensemble Learning, Evolutionary Computation, Hidden Markov Models, Inductive Learning, Naive Bayes, Sequential Pattern Mining, and Support Vector Machine are just a few of the machine learning and data mining techniques used in cyber security that are discussed in the paper. Making effective recommendations is particularly difficult due to the rich and complex nature of the approaches stated. It can be challenging and time-consuming to train representative data for machine learning and data mining. Future developments include the creation of quicker incremental learning models, which would have a significant impact on the field of cyber security.

Using decision trees to improve signature-based intrusion detection [3] paper conveys that Attacks are typically matched with their signatures by intrusion detection systems, which are the norm. Christopher Kruegel and Thomas Toth concentrates on decision tree creation using signatures, which can be used to detect malicious events with the least amount of redundant comparisons. The method that is suggested in the research also makes use of the Rule clustering method to group various attack data types into better trees with fewer redundant comparisons. The fact that there are always more rules and that the amount of data keeps growing over time was one of the biggest problems. SNORT network intrusion detection solutions were outperformed by the Decision Trees technique in terms of speed and performance. Other

intrusion detection systems, such as host- and network-based firewalls and packet filters, can also benefit from using decision trees.

Applications of Machine Learning in Cyber Security [4] by Vitaly Ford and Ambareen Siraj discusses numerous learning applications in the field of Cyber Security. It brings attention to the growing need for cyber security and how, as threats rise, it is becoming increasingly crucial to use machine learning in cyber security. Phishing Detection, Network Intrusion Detection, Authentication with Keystroke Dynamics, Breaking Human Interaction Poofs (CAPTCHAs), Cryptography, Social Network Spam Detection, Smart Data Profiling, and Security of Machine Learning are the applications of machine learning in various cyber security fields that are addressed. Machine learning still has a tonne of potential applications in information security. The work focuses primarily on enhancing the efficiency, performance, and speed of machine learning algorithms in the domain of cyber security.

Machine learning and cyber security [5] proposed by Rishabh Das and Thomas H. Morris in 2017. Rishabh Das and Thomas H. Morris conduct various machine learning algorithms, like Bayesian networks, decision trees, etc., with datasets that they've acquired using various raw data sources. These raw data sources include NetFlow, network packets, etc. These data are fed into various algorithms along with other processes, such as cleaning. Das and Morris conclude that anomaly detection is best done with a clustering algorithm, and to reduce the training time, random forest can be used as it yields the least time, whereas the J-48 algorithm yields a higher training time.

Artificial Neural Networks for Misuse Detections [6] published in 1998. Through this paper James Cannady talks in this paper about how artificial neural networks have the capability to take incomplete, limited, and nonlinear data to identify and classify network activity. Cannady goes through the general neural network and suggests an alternative to the existing methods. Cannady shows how neural networks are the best method for combining machine learning and intrusion detection systems, along with the challenges faced while applying this methodology. The challenges include inadequate data, and once the layers are fixed, the nature of the least changes along with other inputs provided. Although these are threats to the system's accuracy, tests gave the desired result of an RMS of 0.0 and a correlation of 1.

Intrusion Signature Creation via Clustering Anomalies [7] (2008) proposed by Gilbert R. Hendry et al. through this paper look at the clustering algorithm SLCT, which yields highly positive results. The method used by Hendry could

easily separate malicious activity from offline as well as real-time online data. This paper also looks at signature-based IDS, which uses signatures to detect models. Hendry uses the KDD dataset along with modifications to fit the model. The code is emulated, and the performance matrix is recorded. Hendry concludes that IDS and clustering go hand in hand for effective decision-making.

A Review on Cyber Security Datasets for Machine Learning Algorithms [8] (2017) authored by Ozlem Yavanoglu and Murat Aydos. Ozlem Yavanoglu et al. in this paper highlight the dataset used for AI and ML techniques, which are the sources for analysing network traffic and ultimately detecting abnormalities. This paper examines seven distinct datasets as well as two distinct techniques, namely machine learning and artificial intelligence. Yavanoglu explores the core concepts like how to provide integrity and the common challenges like DOS, loss of information, etc. Similar studies are also explored, and datasets are compared on the basis of their pros and cons.

A Survey on Machine Learning Techniques for Cyber Security in the Last Decade [9] published in 2020 proposed by Kamran Shaukat, Suhui Luo, Vijay Varadharajan, Ibrahim A. Hameed, Min Xu conveys that Cyber security has become a matter of concern globally in achieving enhancements in security measures to detect and react against cyberattacks. Our review here has revealed a rapidly growing interest in machine learning and cyber security in the academics and industry, which has resulted in a growing number of publications, particularly in the last decade. This paper bridges the gap between ML techniques and threats to computer networks and mobile communication by presenting a comprehensive survey of the crossovers between the two areas. This survey presents the literature review on machine learning techniques for intrusion detection, spam detection, and malware detection on computer networks and mobile device in the last decade. Various criteria such as detection rate, time complexity, classification time to detect new and zero-day attacks, and accuracy of an ML model should be considered while selecting a particular model to detect a cyberattack.

Cybersecurity data science: an overview from machine learning perspective [11] published in 2020 proposed by Iqbal H. Sarker, ASM Kayes, Shahriar Badsha, Hamed Alqahtani, Paul Watters, Alex Ng. Overall, this paper aimed not only to discuss cybersecurity data science and relevant methods but also to discuss the applicability towards data-driven intelligent decision making in cybersecurity systems and services from machine learning perspectives. It also discusses how it can impact security data, both in terms of extracting

insight of security incidents and the dataset itself. It aimed to work on cybersecurity data science by discussing the state of the art concerning security incidents data and corresponding security services. Based on the knowledge, a generic multi-layered framework of cybersecurity data science model is provided based on machine learning techniques, where the data is being gathered from diverse sources, and the analytics complement the latest data driven patterns for providing intelligent security services. It specifically focuses on extracting insights from security data, from setting a research design with particular attention to concepts for data-driven intelligent security solutions.

Survey on Applications of Deep Learning and Machine Learning Techniques for Cyber Security [12] published in 2020 proposed by IMohammed I. Alghamdi. The paper presents that machine learning and deep learning methods have a high potential to address and solve the cybersecurity problems. The advanced threats of cybersecurity posed to the organizations and individuals can be solved with high efficiency by using machine learning and deep learning methods. The machine learning methods like decision trees, SVM, and KNN are most common and are being further studied to enhance their effectiveness in solving the problems of cybersecurity. However, deep learning methods like DBN, RNN, and CNN are more advanced than machine learning methods where CNN being the most effective as it offers the highest accuracy of intrusion detection and prevention of cyberattacks. Also, they offer high effectiveness in fighting cybersecurity threats like intrusion, malware, and theft. Nevertheless, these methods still have some limitation like complex training procedure, inefficiencies in fighting novel threats, and complex structure of data. Therefore, more research is needed in these areas to enhance further, the efficiencies of the machines learning and deep learning methods in cybersecurity.

Cybersecurity Threats and Their Mitigation Approaches Using Machine Learning [13] published in 2022 proposed by Mostofa Ahsan Rahul Gomes Md Minhaz Chowdhury Nafiz Rifat Jayden F Connolly conveys that the primary objective of applying machine learning in cybersecurity is to make the process of malware detection more actionable, scalable and effective than traditional approaches, which require human intervention. The detection of hidden trends and insights from network data and building of a corresponding data-driven machine learning model to prevent these attacks is vital to design intelligent security systems. In this survey, the focus is on the machine learning techniques that have been implemented on cybersecurity data to make these systems secure.

Cybersecurity Threats and Their Mitigation Approaches Using Machine Learning [14] published in 2020 proposed by R. Geetha¹ · T. Thilagam¹ presents Detection of the attacks is of more significant in today 's cyber world scenario. methods have been preferred by researchers across different disciplines for providing solutions to their problems. use of various ML/DL methods for the detection of different categories of attacks has been presented. various platforms and tools used for implementing DL/ML methods are explored and the security solutions for the different, these techniques have been widely Cyber security is gaining insight nowadays with the increased internet usage and wide variety of security attacks. are focused and the security solutions to the different categories of attacks are discussed and summarized.

Machine learning techniques applied to cybersecurity [15] published in 2018 proposed by Javier Mart'inez Torres¹ involves Machine learning techniques are a set of mathematical models to solve high non-linearity problems of different topics: prediction, classification, data association, data conceptualization. In this work, the authors review the applications of machine learning techniques in the field of cybersecurity describing before the different classifications of the models based on (1) capabilities of machine learning techniques are to be regarded, but authors focus on prediction and classification, highlighting the possibilities of improving the models in order to minimize the error rates in the applications developed and available in themselves in the field of machine learning techniques within cybersecurity.

On the Effectiveness of Machine and Deep Learning for Cyber Security [10] published in 2018 proposed by Giovanni Apruzzese, Michele Colajanni, Luca Ferretti, Alessandro Guido, Mirco Marchetti mentions that Machine and deep learning approaches are increasingly employed for multiple applications and are being adopted also for cyber security, hence it is important to evaluate when and which category of algorithms can achieve adequate results. This paper analyses these techniques for three relevant cyber security problems: intrusion detection, malware analysis and spam detection. It initially proposes an original taxonomy of the most popular categories of ML algorithms and show which of them are currently applied to which problem. All approaches are vulnerable to adversarial attacks and require continuous re-training and careful parameter tuning that cannot be automatized. Moreover, especially when the same classifier is applied to identify different threats, the detection performance is unacceptably low; a possible mitigation can be achieved by using different ML classifiers for detecting specific threats. Our takeaway is that machine learning techniques can support the security operator activities and automate some tasks, but pros and cons must be known.

XI. IMPORTANT DATASETS FOR CYBER SECURITY IN MACHINE LEARNING

Data is of utmost relevance in ML approaches. A machine learning researcher must have a thorough understanding of the data set before conducting any kind of analysis. Second, network data such as packet capture (pcap), NetFlow, and other raw data cannot be used directly by ML analysis. Preprocessing is necessary in order to use the data with ML tools like WEKA, R, and RapidMiner. As a result, researchers using ML analysis on a customized system must be familiar with the methods for gathering and preparing data. We'll go over some fundamental details about the various data formats as well as a few well-known tools for acquiring network data in this part.

We'll have a brief look at the network data:

A. Packet Level Data

Transmission Control Protocol (TCP), User Datagram Protocol (UDP), Internet Control Message Protocol (ICMP), Internet Gateway Management Protocol (IGMP), and other commonly used protocols are among the 144 IPs that the Internet Engineering Task Force (IETF) has specified. The Internet's packet network traffic is created by user programmes that implement these protocols. An application programming interface (API) known as pcap is used to record the network packets that are sent and received at the computer's physical interface (such as its Ethernet port). The front-end packet capture software libraries for several network tools, such as protocol analysers, packet sniffers, network monitors, network IDSs, and traffic generators, are Libpcap and WinPCap (the Unix and Windows versions, respectively). Tcpdump, Wireshark, Snort, and Nmap are a few well-known applications that consume pcap data.

B. Netflow Data

Cisco initially debuted NetFlow as a router capability. As IP network traffic enters and exits the interface, the router or switch has the capability to record it. A network flow, according to Cisco's NetFlow version 5, is a collection of unidirectional packets that all have the same seven packet attributes: ingress interface, source IP address, destination IP address, IP protocol, source port, destination port, and IP type of service. A NetFlow Exporter, a NetFlow Collector, and an Analysis Console make up the logical NetFlow architecture. Currently, NetFlow is available in ten different versions. Versions 1 through 8 are comparable, while version 9 introduces significant changes to NetFlow.

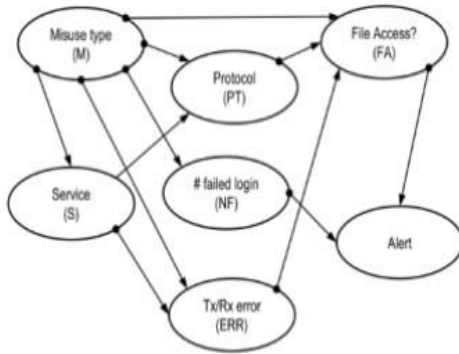
C. Public Data Sets

This includes data such as DARPA1998, DARPA1999, KDD1999, KDD1998. These sets define many types of attacks like DOS, U2R, R2L, etc.

XII. MACHINE LEARNING TECHNIQUES FOR CYBER SECURITY

A. Bayesian Network

Bayesian networks are based on conditional probabilities and dependencies of one variable on another through a



File Access state input variables and values	P(FA = True)	P(FA = False)
M=R2H, PT=NSF, ERR=0	0.95	0.05
M=R2H, PT=FTP, ERR=0	0.99	0.01
M=Probe, PT=none, ERR=50%	0.80	0.20
M=Probe, PT=PING, ERR=0	0.50	0.50
M=DoS, PT=POP, ERR=100%	0.80	0.20
M= DoS, PT=HTTP, ERR=50%	0.90	0.10

Fig. 1. Bayesian Network to identify real life attacks [1] directed acyclic graph where each child node is dependent on their parent node (each node represents state of conditional probability).

Bayesian Networks are used for anomaly detection and see-through known attack signatures and patterns.

In Figure 1, you can see a real-life version of using Bayesian Network to identify attacks

[1] Jami used KDD 1999, which had 9 attributes to model his Intrusion Detection System. In normal scenarios, the performance was 88% and 89% during attack scenarios.

B. Artificial Neural Networks (ANNs)

Neural Networks are layer layout structures where each node in each layer behave like human brain neurons. Each output from one layer in neural network acts as input to another layer. This process is repeated till last layer gives an output.

The main disadvantage of ANNs is the huge learning time and high requirement of resources.

[6] Canady has described an ANN model which uses multicategory classifier to detect anomalies referred as RealSecure network monitor. To train this system 3000 attacks were simulated by Internet Scanner and Satan. The testing phase showed an accuracy of 93%. The error rates were 0.058 and 0.070 during testing and transmission and testing phase. RealSecure Network Monitor classified data traffic as malicious or normal, there was no middle ground between them.

C. Decision Trees

Decision Trees are trees which classify data on the basis of categorical values to definite decisions. Various algorithms like ID3 and C4.5 are used to construct Decision trees. SNORT rules are a series of rules that help in defining malicious network activity. These rules can be used to construct a decision tree with suitable algorithm to develop Intrusion Detection System Softwares. One great example of this is how [3]Kreugal and Toth built an Intrusion Detection System and optimized it by replacing 150 SNORT rules and an intelligent variant of ID3 algorithm to develop the Decision Tree. Rule clusters were used to replace the SNORT rules which helped in increasing the speed of the Intrusion Detection System. This Decision Tree developed by clustered rules was implemented by DARPA (Defence Advanced Research Projects Agency), USA in 1999. In comparison with Decision Trees developed on SNORT rules, it reached a maximum speed of 105%.

D. Clustering

Clustering is an unsupervised learning method to group up the data. The data can be obtained from any audit data or data from previous attacks on systems.

[7] Hendry has demonstrated the use of clustering in real-time signature detection. He employed Simple Logfile Clustering Tool to create normal and anomalous network traffic. This traffic undergoes 2 clustering schemes: first for detection of normal and attack scenarios and second to determine normal traffic in a supervised manner.

[7] Hendry has used the KDD dataset to validate his model where he achieved an accuracy of 70% to 80% for unknown attacks.

ML algorithms like Bayesian Networks and Decision Tree usually are ineffective in unknown attacks as they are highly dependent on the conditions set by previous attack data, but clustering has a better accuracy in detecting unknown attacks.

XIII. APPLICATIONS OF MACHINE LEARNING IN CYBER SECURITY

A. Phishing Detection

It is aimed at stealing sensitive information See Figure 2 for its different type.

B. Network Intrusion Detection

Network Intrusion Detection (NID) systems are used to identify malicious network activity. Many intrusion detection systems are specifically based on machine learning techniques. Figure displays the performance comparison of different algorithms including the proposed one

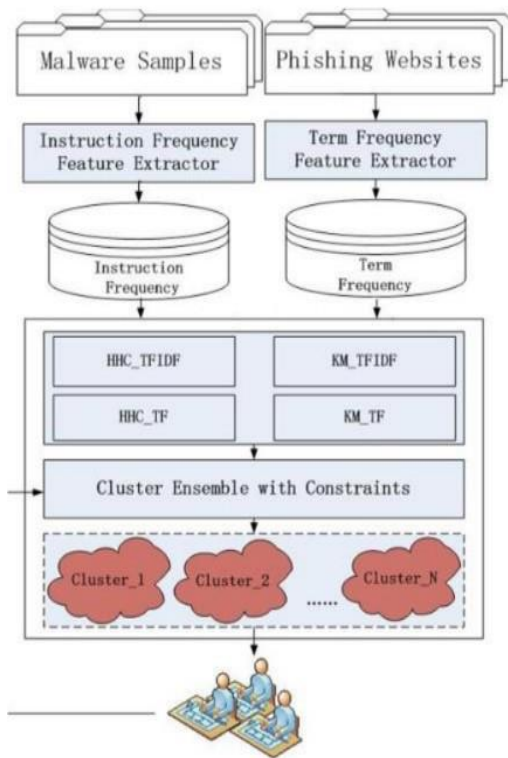


Fig. 2. Architecture of their Automatic Categorization System (ACS) [4]

Fig. 3. Different types of solutions for phishing detection [4]

C. Social Network Spam Detection

For evaluating the performance of the proposed machine learning system, the researchers examined MySpace and Twitter networks. Various legitimate user accounts were

created in both social networks and data were collected over several months. Deceptive spam profiles, like click traps, friend

Detective Solutions	Preventive Solutions	Corrective Solutions
1. Monitors account life cycle	1. Authentication	1. Phishing site takedown
2. Brand monitoring	2. Patch and change management	2. Forensics and investigation
3. Disables web duplication	3. Email authentication	
4. Performs content filtering	4. Web application security	
5. Anti-Malware		
6. Anti-Spam		

infiltrators, duplicate spammers, promoters, and phishers were

manually singled out into several groups. The SVM was fed with the data (for MySpace: 388 legitimate profiles and 627 deceptive spam profiles; for Twitter: 104 legitimate profiles, 61 spammers' and 107 promoters' profiles. Results demonstrate spam precision to be 70% for MySpace and 82% for Twitter.

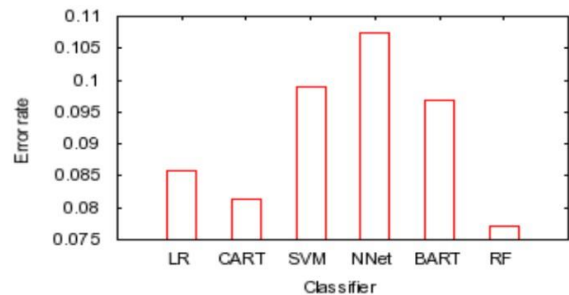


Fig. 4. Different error rates of classification for phishing detection [4]

D. Security of Machine Learning

There are two types of defences against exploratory attacks, in which an attacker can create an evaluation distribution that the learner predicts poorly. For defending against this type of attack, a defender can limit access to the training procedure and data to make it harder for an attacker to apply reverse engineering

Classifier	Precision	Recall	F1
LR	95.11 %	82.96 %	88.59%
CART	92.32 %	87.07 %	89.59 %
SVM	92.08 %	82.74 %	87.07 %
NNet	94.15 %	78.28 %	85.45 %
BART	94.18 %	81.08 %	87.09 %
RF	91.71 %	88.88 %	90.24 %

Fig. 5. Different rates of classification for phishing detection “Logistic Regression (LR), Classification and Regression Trees (CART), Bayesian Additive Regression Trees (BART), Support Vector Machines (SVM), Random Forests (RF), and Neural Networks (NNets) [4]

E. Cryptography

Researchers have shown that it is computationally infeasible to perform some attacks on artificial neural networks. Basically, on the training stage two neural networks start with random weight vectors and receive an arbitrary identical input sequence every cycle. And after a short period of time the corresponding weight vectors of both neural networks become identical.

XIV. RESULT AND DISCUSSION

The machine learning security corporations typically deploy train methods on giant information sets to “learn” what to look out for and the way to react to totally different things on networks. Machine learning is much too powerful in its title, though, and approach could be a natural suited antivirus defence and malware scanning:

- Machine learning approach can facilitate US to stop sensitive information leaks, corporate executive intrusion detection system and malware detection.
- Machine learning is more and more permitting businesses these days to work with higher potency, accuracy, agility, and intelligence.
- This approach could facilitate to unravel the safety problems.
- Machine learning rule will discover and determine any new uncommon pattern or behaviour that may arise from intrusions from outside.
- Security holes caused by allowances created to users or programmers or directors ought to be infatuated a lot of care.

Although many machine algorithms fit for efficient category of cyber security attacks detection and prevention such as for Phishing Detection, in our case formed through analysis of certain Literature reviews on related research papers, Random Forest Algorithm proved to be more efficient with less error

rates and high percentage of F1 score of 90% as compared to other algorithms.

For Network Intrusion Detection, Unified Detection or two stage pruning method emerged as most efficient method as compared to other algorithms with high detection rate of 97%. For Social Network Spam Detection, Bayesian Network can be the most contributing method as Bayesian Networks are used for anomaly detection and see-through known attack signatures and patterns, which is more suitable to the type of attack as compared to other methods.

For Cryptography Artificial Neural Networks can be used as the technique to cope up attacks against cryptographic situations

NID	Detection Rate	ACC	FP	FN
Unified detection (w/ two-stage rule pruning)	97.75%	94.91%	2.01%	2.05%
Unified detection (w/o two-stage rule pruning)	95.79%	90.17%	4.41%	3.75%
GNP-based anomaly detection	86.89%	—	18.4%	0.75%
GNP-based misuse detection	94.71%	—	3.95%	8.54%
Genetic programming	90.83%	—	0.68%	—
Decision trees	—	89.70%	—	—
Support vector machines	95.5%	—	1.0%	—

although at some point it is computationally infeasible to perform some attacks on artificial neural networks.

Fig. 6. Performance Comparison of different Network Intrusion Detection Systems [4]

XV. CONCLUSION AND FUTURE SCOPE

Motivated by the growing significance of cybersecurity and data science, and machine learning technologies, in this paper, we have discussed how cybersecurity data science applies to data-driven intelligent decision making in smart cybersecurity systems and services. We also have discussed how it can impact security data, both in terms of extracting insight of security incidents and the dataset itself. We aimed to work on cybersecurity data science by discussing the state of the art concerning security incidents data and corresponding security

services. We also discussed how machine learning techniques can impact in the domain of cybersecurity, and examine the security challenges that remain. In terms of existing research, much focus has been provided on traditional security solutions, with less available work in machine learning technique-based security systems. For each common technique, we have discussed relevant security research. The purpose of this article is to share an overview of the conceptualization, understanding, modelling, and thinking about cybersecurity data science. One of the key issues for national and international security is the safety of computer systems from cyber-attacks. With numerous machine learning techniques, a survey on security concerns has been presented in this article. Several datasets have been employed to conduct various researches. In addition, a significant role in protection of computer systems is performed by machine learning and artificial intelligence. The literature review of ML/DL and DM methods used for cyber is explained by this paper. The example papers explaining various ML/DL and DM techniques in the cyber domain were carefully found and widespread classes of various datasets have been defined along with their advantages and disadvantages. We aim to generate new dataset in the future, which will be open to all.

REFERENCES

- [1] F. Jemili, M. Zaghdoud, and A. Ben, "A framework for an adaptive intrusion detection system using Bayesian network," *Intelligence and Security Informatics*, IEEE, 2007.
- [2] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," in *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153-1176, Second quarter 2016, doi: 10.1109/COMST.2015.2494502
- [3] C. Kruegel and T. Toth, "Using decision trees to improve signature-based intrusion detection," *Proceedings of the 6th International Workshop on the Recent Advances in Intrusion Detection*, West Lafayette, IN, 2003, pp. 173-191
- [4] V. Ford, and A. Siraj, "Applications of Machine Learning in Cyber Security," in the 27th International Conference on Computer Applications in Industry and Engineering (CAINE), 2014.
- [5] R. Das and T. H. Morris, "Machine Learning and Cyber Security," 2017 International Conference on Computer, Electrical & Communication Engineering (ICCECE), 2017, pp. 1-7, doi: 10.1109/ICCECE.2017.8526232.
- [6] J. Cannady, "Artificial neural networks for misuse detection," *Proceedings of the 1998 National Information Systems Security Conference*, Arlington, VA, 1998, pp. 443-456
- [7] R. Hendry and S. J. Yang, "Intrusion signature creation via clustering anomalies," *SPIE Defense and Security Symposium*, International Society for Optics and Photonics, 2008
- [8] O. Yavanoglu and M. Aydos, "A review on cyber security datasets for machine learning algorithms," 2017 IEEE International Conference on Big Data (Big Data), 2017, pp. 2186-2193, doi: 10.1109/BigData.2017.8258167.
- [9] K. Shaukat, S. Luo, V. Varadharajan, I. A. Hameed and M. Xu, "A Survey on Machine Learning Techniques for Cyber Security in the Last Decade," in *IEEE Access*, vol. 8, pp. 222310-222354, 2020, doi: 10.1109/ACCESS.2020.3041951.
- [10] G. Apruzzese, M. Colajanni, L. Ferretti, A. Guido and M. Marchetti, "On the effectiveness of machine and deep learning for cyber security," 2018 10th International Conference on Cyber Conflict (CyCon), 2018, pp. 371-390, doi: 10.23919/CYCON.2018.8405026.
- [11] Sarker, I.H., Kayes, A.S.M., Badsha, S. et al. Cybersecurity data science: an overview from machine learning perspective. *J Big Data* 7, 41 (2020). <https://doi.org/10.1186/s40537-020-00318-5>
- [12] Alghamdi, Mohammed I. "Survey on Applications of Deep Learning and Machine Learning Techniques for Cyber Security." *International Journal of Interactive Mobile Technologies* 14.16 (2020).
- [13] Ahsan, Mostofa, et al. "Cybersecurity Threats and Their Mitigation Approaches Using Machine Learning—A Review." *Journal of Cybersecurity and Privacy* 2.3 (2022): 527-555.
- [14] Geetha, R., Thilagam, T. A Review on the Effectiveness of Machine Learning and Deep Learning Algorithms for Cyber Security. *Arch Computat Methods Eng* 28, 2861-2879 (2021). <https://doi.org/10.1007/s11831-020-09478-2>
- [15] Martinez Torres, J., Iglesias Comesana, C. & Garcia-Nieto, P.J. Review: machine learning techniques applied to cybersecurity. *Int. J. Mach. Learn. & Cyber.* 10, 2823-2836 (2019).